C#/.NET · UNIDAD 2 · SQL SERVER

01 / CRUD y validación

Preparado para Eric Escobar
Sesión del plan: viernes 23 de octubre de 2026, 4 horas.
Recuperación iniciada: domingo 25 de octubre de 2026.

Referencia: Cronograma_CSharp_NET.pdf, página 2, Semana 1, fila del viernes 23/10: U2 SQL Server, CRUD y validación. La distribución de tiempos se toma de la página 1. [P]

Objetivo de esta sesión
Agregar, consultar, modificar y eliminar registros en ClubPadel; observar cuatro rechazos de integridad y explicar qué regla protege cada uno.

Desde dónde continuamos

La sesión SELECT y filtros quedó cerrada el 22/10: **12 de 12 consultas verificadas**. Ya existen las tres tablas, sus relaciones y 15 filas. Ahora aplicamos las consultas a un ciclo de altas, cambios y bajas; las tablas originales sirven como punto de partida.

Qué deberías poder hacer al terminar

- Escribir INSERT, UPDATE y DELETE con datos y filtros adecuados.
- Comprobar el resultado con SELECT y con las filas afectadas de SSMS.
- Explicar un duplicado de PK, un NULL prohibido y dos conflictos de FK.

Las cuatro horas previstas

Bloque	Tiempo	Trabajo
Repaso	20 min	Modelo y datos confirmados; páginas 2 y 8.
Teoría	45 min	Conceptos y ejemplos; páginas 3 a 13, por bloques.
Ejercicios	110 min	C01 a C04: alta, reserva, cambio y bajas.
Proyecto	45 min	C05 a C08: cuatro pruebas de integridad en ClubPadel.
Corrección y registro	20 min	Revisar evidencias y guardar el punto de continuidad.

Total: 240 minutos. Las pausas quedan fuera del cómputo. Si queda una actividad en curso, se registra para retomarla sin añadir una quinta hora. El avance se obtiene de actividades revisadas, no del tiempo transcurrido.

APUNTE TEÓRICO · SQL SERVER

02 / El punto de partida: ClubPadel

Estos son los datos confirmados al cerrar la sesión anterior. Una **fila** es un registro; una **columna** contiene un dato de ese registro. Consultalos para anticipar y verificar cada operación.

Alumno

IdAlumno	Nombre	Edad
1	Eric	29
2	Carlos	34
3	Leo	15
4	Maria	30
5	Cesar	45

Clase

IdClase	Nombre	Cupo
1	Septima	8
2	Sexta	8
3	Quinta	6
4	Cuarta	4
5	Tercera	4

Reserva

IdReserva	IdAlumno	IdClase
1	3	2
2	4	1
3	1	3
4	5	4
5	2	5

La reserva 3 pertenece a Eric (IdAlumno 1) y a Quinta (IdClase 3). La fila identifica una inscripción concreta. Un mismo alumno o una misma clase pueden participar en varias reservas.

Conservar el avance
Usaremos las tablas existentes. Las altas y bajas de práctica se comprobarán por identificador. Si tus datos actuales difieren de esta referencia, compartí el resultado antes de continuar con una operación que dependa de ellos.

APUNTE TEÓRICO · SQL SERVER

03 / CRUD y preparación en SSMS

CRUD reúne cuatro operaciones sobre registros. Las letras provienen de Create, Read, Update y Delete. En esta sesión, Create significa dar de alta una fila mediante INSERT. La lectura con SELECT permite comprobar cada cambio. [S1-S3]

Operación	SQL	Ejemplo de uso
Alta / Create	INSERT	Registrar un alumno nuevo.
Lectura / Read	SELECT	Ver si el alumno quedó registrado.
Cambio / Update	UPDATE	Corregir su edad.
Baja / Delete	DELETE	Quitar su reserva o su registro de práctica.

Dónde escribir y ejecutar

Abrí **SQL Server Management Studio (SSMS)**, conectate al servidor y abrí **New Query**. Usá un archivo nuevo para esta sesión y guardalo como **CRUD_2026-10-23.sql**. Escribí al inicio:

```
USE ClubPadel;
GO
```

USE ClubPadel selecciona la base de trabajo. **GO** separa bloques que SSMS envía al servidor. Seleccioná la instrucción que querés ejecutar y presioná F5 o Execute. Los comentarios que empiezan con -- sirven para identificar C01, C02, etc.

Cómo reconocer una operación correcta

- **Antes:** mirá los datos con SELECT y anticipá qué fila debería cambiar.
- **Durante:** ejecutá una instrucción de alta, cambio o baja y leé Messages.
- **Después:** consultá la misma clave y compará el resultado esperado con Results.

El mensaje habitual **(1 row affected)** indica una fila afectada. Un UPDATE o DELETE con cero coincidencias puede mostrar **(0 rows affected)** sin error de integridad. Confirmá siempre los valores con SELECT; el mensaje por sí solo no explica qué dato quedó guardado.

Ejemplos y práctica
Los ejemplos resueltos de las páginas 4 a 7 forman una secuencia con Nora y el identificador 90. Se estudian en ese orden. Tus actividades usarán otros datos y se entregarán de a una en la conversación; sus soluciones se reservan para después de tu intento.

APUNTE TEÓRICO · SQL SERVER

04 / INSERT: registrar un alumno

INSERT INTO agrega una fila a una tabla. Se utiliza cuando aparece un registro nuevo: por ejemplo, una persona que se incorpora al club. Indicamos las columnas y después sus valores, en el mismo orden. [S1]

Problema resuelto: dar de alta a Nora, de 26 años

Supongamos que el IdAlumno 90 está libre en los datos de la página 2. En SSMS, sobre ClubPadel, la instrucción es:

```
INSERT INTO Alumno (IdAlumno, Nombre, Edad)
VALUES (90, N'Nora', 26);
```

Parte	Qué hace y por qué
INSERT INTO Alumno	Elige Alumno como tabla donde se agrega la fila.
(IdAlumno, Nombre, Edad)	Declara el orden de las columnas que vamos a completar.
VALUES	Introduce los datos del registro nuevo.
(90, N'Nora', 26)	Asigna 90 a IdAlumno, Nora a Nombre y 26 a Edad.
Punto y coma ;	Cierra la instrucción.

Los números se escriben directamente. **N'Nora'** es un valor de texto entre comillas simples; el prefijo N indica Unicode, adecuado para NVARCHAR y caracteres como tildes o ñ. En nuestras tablas los identificadores se proporcionan explícitamente.

Comprobar el alta

```
SELECT IdAlumno, Nombre, Edad
FROM Alumno
WHERE IdAlumno = 90;
```

IdAlumno	Nombre	Edad
90	Nora	26

El INSERT debería informar una fila afectada. La consulta de comprobación devuelve exactamente ese alumno. El WHERE elige su identificador; SELECT muestra lo que efectivamente quedó guardado.

Decisión clave
La PK debe estar libre antes del alta. Si repetís el mismo INSERT después de que funcionó, intentás crear otra fila con IdAlumno 90 y aparece un duplicado. En la página 9 estudiamos ese rechazo.

APUNTE TEÓRICO · SQL SERVER

05 / INSERT en una tabla relacionada

Una reserva vincula un alumno con una clase. Para darla de alta escribimos tres números, pero cada columna cumple una función distinta. Las claves foráneas comprueban que el alumno y la clase existan. [S4]

Problema resuelto: reservar Sexta para Nora

Continuamos con Nora ya registrada como IdAlumno 90. Sexta existe como IdClase 2. Elegimos un IdReserva nuevo, también 90; que ambos números coincidan en este ejemplo no los convierte en la misma clave.

```
INSERT INTO Reserva (IdReserva, IdAlumno, IdClase)
VALUES (90, 90, 2);
```

Columna	Valor	Interpretación
IdReserva	90	Identifica esta reserva; es la PK de Reserva.
IdAlumno	90	Referencia al alumno Nora; es una FK en Reserva.
IdClase	2	Referencia a Sexta; es otra FK en Reserva.

Primero se registra el alumno y después su reserva porque la segunda operación depende de la primera. La clase ya estaba creada, así que su identificador puede usarse directamente.

Comprobación

```
SELECT IdReserva, IdAlumno, IdClase
FROM Reserva
WHERE IdReserva = 90;
```

IdReserva	IdAlumno	IdClase
90	90	2

Esperamos una fila afectada y una fila en la consulta. Para interpretar el resultado, leemos cada columna: reserva 90, alumno 90, clase 2. La posición de la fila en la grilla no reemplaza a ninguno de esos identificadores.

Cuándo puede repetirse un número

Otra reserva de Nora usaría un **IdReserva diferente** y conservaría **IdAlumno 90**. Otra persona puede reservar Sexta y usar IdClase 2. Las FK pueden repetirse; cada valor debe apuntar a una fila existente.

Elegir la columna según la pregunta
Si buscás la reserva concreta, filtrás por IdReserva. Si buscás las reservas de Nora, filtrás por IdAlumno. Si buscás las de Sexta, filtrás por IdClase. La elección responde al dato que querés localizar.

APUNTE TEÓRICO · SQL SERVER

06 / UPDATE: modificar un registro

UPDATE cambia valores de filas existentes. Se utiliza para corregir o actualizar información, como una edad. **SET** indica qué columna recibe qué valor; **WHERE** identifica las filas alcanzadas por el cambio. [S2]

Problema resuelto: corregir la edad de Nora a 27

Primero localizamos el registro con la misma condición que usaremos para actualizarlo:

```
SELECT IdAlumno, Nombre, Edad
FROM Alumno
WHERE IdAlumno = 90;
```

IdAlumno	Nombre	Edad
90	Nora	26

```
UPDATE Alumno
SET Edad = 27
WHERE IdAlumno = 90;
```

Línea	Motivo
UPDATE Alumno	Modificamos una fila de Alumno.
SET Edad = 27	Asignamos 27 a la columna Edad de la fila elegida.
WHERE IdAlumno = 90	Elegimos el registro concreto mediante su PK.

Verificar el cambio

```
SELECT IdAlumno, Nombre, Edad
FROM Alumno
WHERE IdAlumno = 90;
```

IdAlumno	Nombre	Edad
90	Nora	27

El resultado esperado es una fila afectada y la edad 27 en la consulta. Nombre e IdAlumno conservan sus valores porque la instrucción solo asignó un nuevo valor a Edad.

El alcance del WHERE
La condición determina qué filas se modifican. Una condición sobre una PK existente elige una sola fila. Si no se escribe WHERE, UPDATE intenta cambiar todas las filas de la tabla. Antes de ejecutar, comprobá que la condición coincide con el registro pedido.

Si IdAlumno 90 no existiera, este UPDATE no crearía un alumno: afectaría cero filas. Para dar un alta usamos INSERT. Para identificar un problema, compará el SELECT previo con la clave que aparece en el WHERE.

APUNTE TEÓRICO · SQL SERVER

07 / DELETE: quitar filas relacionadas

DELETE FROM elimina filas de una tabla. El WHERE determina cuáles. En un club puede representar la cancelación de una reserva. Al eliminar registros relacionados, el orden de las operaciones debe respetar sus FK. [S3-S4]

Problema resuelto: cerrar el ciclo de práctica de Nora

Nora tiene una reserva que la referencia. Primero comprobamos esa relación:

```
SELECT IdReserva, IdAlumno, IdClase
FROM Reserva
WHERE IdAlumno = 90;
```

IdReserva	IdAlumno	IdClase
90	90	2

Paso 1. Quitar la reserva de práctica

```
DELETE FROM Reserva
WHERE IdReserva = 90;
```

DELETE FROM Reserva elige la tabla. **WHERE IdReserva = 90** limita la baja a esa reserva. Esperamos una fila afectada. La inscripción se elimina y Nora todavía existe como alumno.

Paso 2. Quitar el alumno de práctica

```
DELETE FROM Alumno
WHERE IdAlumno = 90;
```

Ahora Nora ya no tiene esa referencia desde Reserva. El DELETE de Alumno debería afectar una fila. La tabla Alumno sigue disponible para los demás registros.

Paso 3. Comprobar las dos bajas

```
SELECT IdReserva, IdAlumno, IdClase
FROM Reserva
WHERE IdReserva = 90;

SELECT IdAlumno, Nombre, Edad
FROM Alumno
WHERE IdAlumno = 90;
```

Ambas consultas devuelven cero filas. Al completar este ejemplo, las tres tablas vuelven a los datos originales de la página 2. Cada baja se verificó por su propia PK.

DELETE sin WHERE intenta eliminar todas las filas de la tabla. En nuestras consignas, localizamos primero el registro con SELECT y usamos su identificador para que el alcance coincida con la baja pedida.

APUNTE TEÓRICO · SQL SERVER

08 / Integridad: las reglas de tus tablas

Una **restricción** es una regla que la base de datos comprueba al recibir una operación. La **integridad de datos** consiste en mantener los datos de acuerdo con esas reglas. Un rechazo puede ser el resultado esperado de una prueba. [S4]

Regla	Qué exige	En ClubPadel
PRIMARY KEY / PK	Un identificador único y obligatorio por fila.	IdAlumno en Alumno, IdClase en Clase e IdReserva en Reserva.
NOT NULL	Que la columna tenga un valor presente.	Nombre y Edad de Alumno; Nombre y Cupo de Clase; ambas FK de Reserva.
FOREIGN KEY / FK	Que la referencia corresponda a una clave existente.	Reserva.IdAlumno apunta a Alumno.IdAlumno; Reserva.IdClase apunta a Clase.IdClase.

Cómo reconocer las reglas en el esquema que ya creaste

```
IdAlumno INT PRIMARY KEY
Nombre NVARCHAR(50) NOT NULL
Edad INT NOT NULL
```

Estas líneas describen las columnas de Alumno dentro de su CREATE TABLE original. Son una referencia de lectura: la tabla ya existe. INT guarda enteros; NVARCHAR(50) es el tipo de texto Unicode que usamos para Nombre.

```
FOREIGN KEY (IdAlumno) REFERENCES Alumno(IdAlumno)
FOREIGN KEY (IdClase) REFERENCES Clase(IdClase)
```

En Reserva, estas reglas vinculan cada FK con su tabla de referencia. Decimos **tabla padre** a la que aporta la fila referenciada y **tabla hija** a la que contiene la FK. Alumno y Clase son padres de Reserva en estas relaciones.

Dos comprobaciones diferentes de una FK

• Al agregar o cambiar una reserva, el alumno y la clase elegidos deben existir.

• Al eliminar un padre, la base comprueba si quedan reservas que lo necesitan.

Tus FK se crearon con el comportamiento predeterminado, sin borrado en cascada. Por eso una baja del padre con reservas existentes se rechaza. Estudiaremos cuatro casos concretos en las páginas siguientes.

Qué vamos a demostrar
No alcanza con leer que hubo un error. Para completar cada prueba, identificá la operación, la columna o relación afectada, la regla incumplida y los datos que permiten comprobarlo.

APUNTE TEÓRICO · SQL SERVER

09 / Error 1: una PK repetida

La PK permite distinguir cada fila. En Alumno ya existe Carlos con IdAlumno 2. Intentar usar nuevamente ese identificador para dar un alta incumple la regla de unicidad. [S4, S6]

Ejemplo resuelto: intentar otra alta con IdAlumno 2

```
SELECT IdAlumno, Nombre, Edad
FROM Alumno
WHERE IdAlumno = 2;
```

IdAlumno	Nombre	Edad
2	Carlos	34

```
INSERT INTO Alumno (IdAlumno, Nombre, Edad)
VALUES (2, N'Prueba', 20);
```

Qué se intenta: crear una fila adicional. **Qué se rechaza:** repetir el valor 2 en la PK IdAlumno. Que Nombre y Edad sean diferentes no vuelve único al identificador.

Qué debería mostrar SSMS

Se espera el error **2627**, referido a una restricción de clave y a un valor duplicado. El nombre concreto de la restricción puede haber sido generado por SQL Server. Buscá también la tabla y el valor que informa el mensaje. [S6]

Cómo comprobar que la fila original continúa

```
SELECT IdAlumno, Nombre, Edad
FROM Alumno
WHERE IdAlumno = 2;
```

IdAlumno	Nombre	Edad
2	Carlos	34

El intento fallido no reemplazó a Carlos por Prueba. INSERT se usa para agregar una fila. Si el propósito fuera corregir un alumno existente, la operación adecuada sería UPDATE con la condición correspondiente.

Cómo corregir la causa

Para un alumno nuevo, elegí una clave disponible y verificá que no esté ocupada. Para un alumno existente, localizá su fila y modificá el dato solicitado. La decisión depende de si se busca un alta o un cambio.

Explicación modelo
Se rechazó el alta porque IdAlumno 2 ya identifica a Carlos. La PK de Alumno exige que cada identificador aparezca una sola vez. La consulta posterior conserva los datos originales.

APUNTE TEÓRICO · SQL SERVER

10 / Error 2: NULL en un dato obligatorio

NULL representa la ausencia de un valor. **NOT NULL** exige que una columna tenga un dato presente. Como Nombre se creó con NOT NULL, no se puede registrar un alumno cuyo nombre sea NULL. [S5]

Ejemplo resuelto: intentar un alta sin nombre

Tomamos un identificador libre, 92, para que el problema a observar sea la ausencia de Nombre. Edad recibe un entero válido.

```
INSERT INTO Alumno (IdAlumno, Nombre, Edad)
VALUES (92, NULL, 20);
```

92 corresponde a IdAlumno. **NULL**, escrito sin comillas, corresponde a Nombre. **20** corresponde a Edad. El rechazo ocurre en Nombre, porque esa columna es obligatoria.

Qué debería mostrar SSMS

Se espera el error **515**, que identifica una columna que no admite NULL. Leé qué columna y tabla aparecen en tu mensaje. La fila nueva no queda incorporada. [S5]

```
SELECT IdAlumno, Nombre, Edad
FROM Alumno
WHERE IdAlumno = 92;
```

Resultado esperado: **cero filas**. La grilla conserva los encabezados.

NULL, texto vacío y cero

Valor	Qué representa	¿Es NULL?
NULL	Ausencia de un valor.	Sí.
N''	Un texto vacío: tiene longitud cero.	No.
N'NULL'	Las cuatro letras N, U, L y L como texto.	No.
0	El número cero.	No.

Con las reglas actuales, NOT NULL por sí solo no impide un texto vacío ni una Edad igual a 0. Para esta prueba necesitamos una ausencia real, expresada con NULL; así identificamos exactamente qué regla se está comprobando.

Cómo corregir la causa

Proporcioná un nombre real en la columna Nombre. Si una columna obligatoria no tiene valor predeterminado y la omitís del INSERT, también puede producirse un rechazo por NULL. Comprobá qué columnas exige el esquema antes del alta.

Explicación modelo
Se rechazó la fila porque Nombre recibió NULL y fue definido como NOT NULL. El identificador estaba libre; el motivo del error fue la ausencia de un dato obligatorio.

APUNTE TEÓRICO · SQL SERVER

11 / Error 3: una FK apunta a alguien inexistente

Una FK conecta una reserva con una fila real de Alumno o Clase. La relación perdería sentido si se permitiera reservar en nombre de un alumno que todavía no existe. [S4]

Ejemplo resuelto: una reserva para IdAlumno 888

En los datos de referencia no existe el alumno 888. La clase 2 sí existe. Usamos IdReserva 91, libre, para que el motivo del rechazo sea la referencia al alumno.

```
SELECT IdAlumno, Nombre, Edad
FROM Alumno
WHERE IdAlumno = 888;
```

Resultado esperado: **cero filas**. La grilla conserva los encabezados.

```
INSERT INTO Reserva (IdReserva, IdAlumno, IdClase)
VALUES (91, 888, 2);
```

El primer número identifica la reserva. El segundo se comprueba contra Alumno.IdAlumno. El tercero se comprueba contra Clase.IdClase. En este caso, el segundo valor no encuentra una fila padre.

Qué debería mostrar SSMS

Se espera el error **547**, referido al conflicto con una restricción FOREIGN KEY. El mensaje indica la relación o tabla implicada. Este número identifica conflictos de restricciones; para interpretar el caso, hay que leer también la operación y el texto del error. [S5]

```
SELECT IdReserva, IdAlumno, IdClase
FROM Reserva
WHERE IdReserva = 91;
```

Resultado esperado: **cero filas**. La grilla conserva los encabezados.

Cómo corregir la causa

Si el alumno debe existir, primero completá su alta y verificá su identificador; después registrá su reserva. Si se escribió mal el número, corregilo usando la clave del alumno real. Elegir cualquier número libre como FK no crea automáticamente al alumno.

Explicación modelo
Se rechazó la reserva porque IdAlumno 888 no aparece en Alumno. La FK de Reserva exige una referencia existente. La clase 2 es válida; la referencia al alumno es la que falla.

APUNTE TEÓRICO · SQL SERVER

12 / Error 4: borrar un padre con reservas

La integridad referencial también se comprueba al quitar un registro padre. En el modelo actual, una reserva necesita conservar su alumno y su clase. Una baja del padre con referencias activas se rechaza. [S3-S4]

Ejemplo resuelto: intentar eliminar a Maria

Maria tiene IdAlumno 4 y conserva la reserva 2. Primero comprobamos la fila que la referencia:

```
SELECT IdReserva, IdAlumno, IdClase
FROM Reserva
WHERE IdAlumno = 4;
```

IdReserva	IdAlumno	IdClase
2	4	1

```
DELETE FROM Alumno
WHERE IdAlumno = 4;
```

DELETE FROM Alumno intenta quitar al padre. **WHERE IdAlumno = 4** localiza a Maria. Aunque esa condición identifica correctamente la fila, la reserva 2 todavía contiene IdAlumno 4.

Qué debería mostrar SSMS

Se espera un conflicto **547** de FK. En este caso la operación es DELETE y el registro padre está siendo usado desde Reserva. Es el mismo número que en el caso anterior, pero se incumple la relación mediante una operación diferente. [S5]

```
SELECT IdAlumno, Nombre, Edad
FROM Alumno
WHERE IdAlumno = 4;
```

IdAlumno	Nombre	Edad
4	Maria	30

Maria sigue registrada y su reserva conserva una referencia válida. Para realizar una baja autorizada de un alumno y sus reservas, se atienden primero los registros hijos, como mostramos con Nora en la página 7.

Cómo explicar este resultado

Identificá la fila padre que se quiso quitar, la reserva que la necesita y la relación que protege esos datos. En esta prueba, observar el rechazo y demostrar que las filas continúan es el resultado solicitado.

Explicación modelo
La baja se rechazó porque la reserva 2 todavía referencia a Maria mediante IdAlumno 4. La FK evita que esa reserva quede apuntando a un alumno inexistente.

APUNTE TEÓRICO · SQL SERVER

13 / Leer resultados y corregir errores

Para revisar una operación, distinguí su intención, el mensaje del servidor y el estado final de los datos. Primero identificá qué se pidió; después comprobá la tabla, las columnas, los valores y el WHERE.

Qué observás	Posible causa	Cómo lo comprobás o corregís
Una fila afectada	La operación alcanzó una fila.	Consultá su PK y verificá los valores esperados.
Cero filas afectadas	El WHERE no encontró una fila.	Ejecutá SELECT con la misma condición y revisá el identificador.
Varias filas afectadas	La condición abarca varios registros.	Compará el conjunto elegido con el alcance pedido.
Error 2627	Un valor de clave está repetido.	Buscá esa PK en su tabla y decidí si corresponde alta o cambio.
Error 515	Una columna obligatoria recibió NULL.	Leé qué columna aparece en el mensaje y completá el dato.
Error 547 al insertar	Una referencia de la reserva no existe.	Consultá el alumno y la clase usados como FK.
Error 547 al borrar	El padre todavía tiene referencias.	Buscá las reservas que lo referencian.
Error cerca de una palabra	Puede haber un problema de sintaxis.	Revisá comas, paréntesis, VALUES, SET y comillas simples.

Errores de escritura frecuentes

• Cada lista de columnas debe corresponder con la lista de valores del INSERT.

• Para varios registros en un INSERT se usa una lista de filas separadas por comas; no se repite VALUES como si fuera otra instrucción aislada.

• UPDATE necesita SET para indicar la asignación. El WHERE limita las filas que reciben ese cambio.

• En estas comparaciones SQL usamos =. Los textos van entre comillas simples y NULL se escribe sin ellas.

Un formato simple para compartir tu evidencia

Enviá: **consulta ejecutada + mensaje o grilla + explicación**. Para un error de integridad agregá qué regla se incumple y una consulta que permita comprobar los datos implicados. El nombre exacto de una restricción puede variar entre instalaciones; interpretamos lo que protege.

Cuando un resultado no coincide
Mantenemos la actividad en curso, identificamos la causa y corregimos una parte por vez. Un error esperado queda validado cuando podés explicar por qué se produjo, no solo cuando aparece texto rojo.

APUNTE TEÓRICO · SQL SERVER

14 / Las ocho actividades de la sesión

El plan original pide altas, cambios, bajas y **cuatro errores de integridad**; no enumera consignas individuales. Esta organización los distribuye en **8 actividades**, que se trabajarán de a una. C01 a C04 ocupan el bloque de ejercicios y C05 a C08 aplican la validación al proyecto.

ID	Actividad y referencia del apunte	Estado inicial
C01	Alta guiada de un alumno y comprobación con SELECT. Página 4.	en curso
C02	Alta autónoma de su reserva y comprobación de las referencias. Página 5.	pendiente
C03	Cambio de un dato con UPDATE, comparando antes y después. Página 6.	pendiente
C04	Bajas con DELETE: reserva y alumno de práctica, en ese orden. Página 7.	pendiente
C05	Prueba y explicación de un duplicado de PK. Página 9.	pendiente
C06	Prueba y explicación de un NULL prohibido. Página 10.	pendiente
C07	Prueba y explicación de una FK hacia un alumno inexistente. Página 11.	pendiente
C08	Prueba y explicación del rechazo al borrar un padre con reservas. Página 12.	pendiente

Cómo se calcula el avance

Progreso = (actividades completadas / 8) x 100. Cada actividad revisada representa 12,5 puntos porcentuales. Dos actividades completas equivalen al 25 %. Una actividad en curso todavía no suma.

Fotografía inicial del progreso
0 % - 0 de 8 actividades completadas. C01 está en curso; C02 a C08 están pendientes. Este PDF conserva el estado inicial. La conversación irá registrando los resultados efectivamente revisados.

Qué necesitamos para marcar una actividad completa

En las operaciones correctas, revisamos la consulta, las filas afectadas, los valores finales y tu explicación. En C04 verificamos las dos bajas antes de sumarla. En las cuatro pruebas de error, comprobamos el rechazo, su causa y el estado posterior de los datos.

Los ejemplos resueltos y la lectura del apunte preparan la práctica. El total se mantiene en ocho actividades salvo que acordemos cambiar el alcance. El 12/12 de SELECT pertenece a la sesión anterior y conserva su cierre independiente.

APUNTE TEÓRICO · SQL SERVER

15 / Resumen y glosario para repasar

Las decisiones principales

- **INSERT:** agrego una fila nueva con una clave disponible y datos completos.
- **SELECT:** compruebo el registro antes y después de una operación.
- **UPDATE + SET + WHERE:** asigno un valor a las filas que elegí.
- **DELETE + WHERE:** elimino las filas solicitadas respetando las referencias.
- **PK, NOT NULL y FK:** explican los cuatro rechazos que vamos a probar.

Glosario

Término	Significado en esta sesión
CRUD	Conjunto de altas, lecturas, cambios y bajas sobre registros.
Registro / fila	Una entrada de una tabla, como un alumno o una reserva.
Columna	Un atributo del registro, como Nombre, Edad o IdClase.
INSERT / VALUES	Instrucción de alta y parte que proporciona los valores.
UPDATE / SET	Instrucción de cambio y parte que asigna los nuevos valores.
DELETE	Instrucción que elimina filas de una tabla.
WHERE	Condición que selecciona las filas de una operación.
PK	Clave primaria que identifica de forma única cada fila.
FK	Clave foránea que referencia una clave existente en otra tabla.
NULL / NOT NULL	Ausencia de valor y regla que exige un valor presente.
Integridad	Cumplimiento de las reglas de los datos y sus relaciones.
Filas afectadas	Cantidad de filas alcanzadas por una operación.

Una pregunta para reconocer la operación
¿Necesito agregar una fila, consultar lo que existe, cambiar un dato o quitar un registro?
Después de elegir la operación, decido la tabla y la condición a partir del pedido concreto.

APUNTE TEÓRICO · SQL SERVER

16 / Referencias y punto de continuidad

[P] Plan original: Cronograma_CSharp_NET.pdf. Página 1: distribución de las cuatro horas. Página 2: sesión del viernes 23/10/2026, U2 SQL Server, CRUD y validación. Los datos y el cierre 12/12 provienen del avance confirmado en esta conversación.

Documentación oficial de Microsoft

[S1] INSERT (Transact-SQL)
Sintaxis del alta, columnas y valores.

[S2] UPDATE (Transact-SQL)
Asignaciones con SET y alcance del WHERE.

[S3] DELETE (Transact-SQL)
Bajas por condición y conflictos con referencias.

[S4] Restricciones de claves principales y externas
Unicidad de la PK e integridad de las FK.

[S5] Errores del motor: 0 a 999
Mensajes 515 y 547.

[S6] Errores del motor: 2000 a 2999
Mensaje 2627 de clave duplicada.

Enlaces consultados el 25/10/2026. Los ejemplos de ClubPadel son propios y sus resultados son los esperados para el esquema y los datos indicados.

Cómo dejamos el registro al finalizar

Dato	Qué vamos a anotar
Actividades	Identificadores C01 a C08 realmente completados y estados.
Evidencia	Consultas, resultados, errores observados y explicación.
Trabajo	SSMS, base ClubPadel y archivo CRUD_2026-10-23.sql si fue guardado con ese nombre.
Pendiente	Duda o paso concreto dentro de la actividad en curso.
Continuidad	Próxima acción exacta, sin sumar avance por la fecha.

Punto de inicio: C01, alta guiada de un alumno. Consultá la teoría de INSERT en la página 4 y esperá la consigna de la conversación. Después de tu intento revisaremos consulta, resultado y razonamiento.